IMPLEMENTASI ALGORITMA *BRUTE FORCE* PADA *TRAVELING SALESMAN PROBLEM* (TSP) UNTUK OPTIMASI RUTE KONSER *WORLD TOUR* TWICE DI INDONESIA

Penyusun:

Puji Astuti Suryaningtyas

(24533867)

ALGORITMA STRATEGIS

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH PONOROGO

2025/2026

BAB I
LANDASAN TEORI

1.1. Traveling Salesman Problem (TSP)

Dalam penyelenggaraan tur konser musik, aspek logistik sering kali menjadi tantangan tersembunyi yang paling kritis. Selain urusan artistik, manajemen tur harus memastikan perpindahan artis, kru, dan peralatan dari satu venue ke venue lain berjalan efisien. Kesalahan dalam menentukan urutan kunjungan kota bukan hanya soal waktu tempuh yang membengkak, tetapi juga berimplikasi pada biaya bahan bakar, akomodasi, dan kelelahan tim. Secara matematis, inti permasalahan ini dikenal sebagai *Traveling Salesman Problem* (TSP), yaitu pencarian rute terpendek untuk mengunjungi sejumlah titik lokasi tepat satu kali sebelum kembali ke titik awal (Link Sumber: https://www.indonesia.travel/gb/en/events/event-detail/babymonster-1st-world-tour/).

Traveling Salesman Problem (TSP) merupakan masalah klasik dalam ilmu komputer dan riset operasi yang sering dijadikan standar untuk menguji efisiensi algoritma optimasi. Permasalahan ini berfokus pada pencarian rute terpendek bagi seorang penjelajah yang harus mengunjungi sejumlah kota tertentu, di mana setiap kota wajib dikunjungi tepat satu kali sebelum kembali ke titik awal. Tujuan utamanya adalah menentukan urutan kunjungan yang meminimalkan total jarak tempuh atau biaya perjalanan.

Secara matematis, TSP dimodelkan menggunakan graf berbobot lengkap $G = (V, E)$, dengan (V) yang merepresentasikan kota dan (E) yang menyatakan jalur penghubung antar kota beserta bobotnya berupa jarak atau biaya. Tantangan fundamental TSP tidak terletak pada pemahaman konsep, melainkan pada pertumbuhan jumlah kemungkinan rute yang meningkat secara drastis seiring penambahan jumlah kota. Untuk n buah kota, terdapat $(n-1)!$ kemungkinan permutasi rute jika arah perjalanan diperhitungkan. Sifat ledakan kombinatorial ini menempatkan TSP dalam kategori masalah *NP-Hard*, yang menyebabkan pencarian solusi pasti (*exact solution*) menjadi sangat berat

secara komputasi saat jumlah kota bertambah besar (Link Sumber: https://terralogiq.com/traveling-salesman/).

1.2. Algoritma Brute Force

Untuk menyelesaikan TSP, salah satu pendekatan paling dasar adalah algoritma *Brute Force*. Sesuai namanya, *Brute Force* hadir sebagai pendekatan paling dasar yang menerapkan prinsip *exhaustive search* yang mencoba seluruh kemungkinan urutan rute untuk menjamin ditemukannya solusi optimal mutlak. Algoritma ini akan membangkitkan setiap urutan permutasi kota yang mungkin, menghitung total jarak untuk masing-masing rute, lalu membandingkannya satu per satu untuk menemukan nilai minimum.

Algoritma *Brute Force* sebagai pendekatan paling dasar yang menerapkan prinsip *exhaustive search* yang mencoba seluruh kemungkinan urutan rute untuk menjamin ditemukannya solusi optimal mutlak. Memang, kompleksitas waktunya yang tumbuh secara faktorial ($O(n!)$) membuat metode ini tidak efisien jika jumlah kota terlalu banyak. Namun, justru karena sifatnya yang menyeluruh, algoritma ini sangat ideal untuk kasus dengan jumlah simpul terbatas, di mana dapat berfungsi sebagai standar validasi logika dan bahan komparasi yang akurat.

Pendekatan ini memiliki karakteristik mendasar yang perlu dipahami. Dari segi kelebihan, logika algoritma ini sangat lurus dan mudah diterjemahkan ke dalam kode program karena hanya mengikuti definisi masalah secara harfiah tanpa memerlukan struktur data yang rumit. Lebih *importantly*, karena tidak ada kemungkinan rute yang dilewatkan selama proses pencarian, solusi yang dihasilkan dijamin 100% merupakan rute terpendek global (*global optimum*). Hal ini menghilangkan risiko terjebak pada solusi lokal (*local optimum*) yang sering kali menjadi kelemahan pada algoritma heuristik (Link: https://jurnal.untan.ac.id/index.php/jbmstr/article/download/9608/9421).

Di sisi lain, keterbatasan utama algoritma ini terletak pada efisiensi komputasinya. Dengan kompleksitas waktu faktorial, kinerja algoritma menurun drastis seiring penambahan input; penambahan satu kota saja akan

melipatgandakan waktu proses sebanyak n kali lipat. Akibatnya, metode ini menjadi tidak praktis untuk masalah skala besar.

1.3. Mekanisme Penyelesaian TSP dengan *Brute Force*

Dalam implementasi tugas besar ini, algoritma Brute Force diterapkan melalui langkah-langkah sistematis berikut:

1. Inisialisasi Data: Program memulai dengan memuat daftar kota yang dipilih pengguna dan matriks jarak antar kota tersebut. Data ini menjadi acuan dasar perhitungan bobot.
2. Pembangkitan permutasi rute yang menghasilkan semua skenario urutan kunjungan yang mungkin. Agar komputasi lebih ringan, titik awal perjalanan dikunci tetap, sehingga algoritma hanya memvariasikan urutan $n - 1$ kota sisanya. Fungsi bawaan C++ next_permutation digunakan di sini untuk menghasilkan variasi rute tersebut secara otomatis.
3. Perhitungan total jarak (evaluasi bobot) untuk setiap urutan rute yang terbentuk. Program menjumlahkan jarak antar kota yang bertetangga dalam urutan tersebut, perhitungan harus mencakup jarak dari kota terakhir kembali ke kota awal untuk membentuk siklus tertutup (*closed loop*).
4. Seleksi rute terpendek dari hasil total jarak dari setiap permutasi yang dibandingkan secara iteratif. Program menyimpan rute dengan total jarak terkecil yang ditemukan sejauh ini sebagai kandidat solusi terbaik.
5. Penyajian hasil setelah seluruh permutasi diperiksa. Program menampilkan urutan kota pada rute terbaik beserta total jarak tempuhnya kepada pengguna.

Pendekatan ini memang menuntut waktu komputasi lebih tinggi untuk data besar, namun hal tersebut sebanding dengan jaminan kepastian yang diberikan, rute yang dihasilkan adalah solusi terbaik yang mungkin ada untuk skenario tersebut. (Link: https://youtu.be/gsYmZdZuFCw?si=oACj2tKPGRW7r9sJ).

BAB II
METODOLOGI DAN SIMULASI

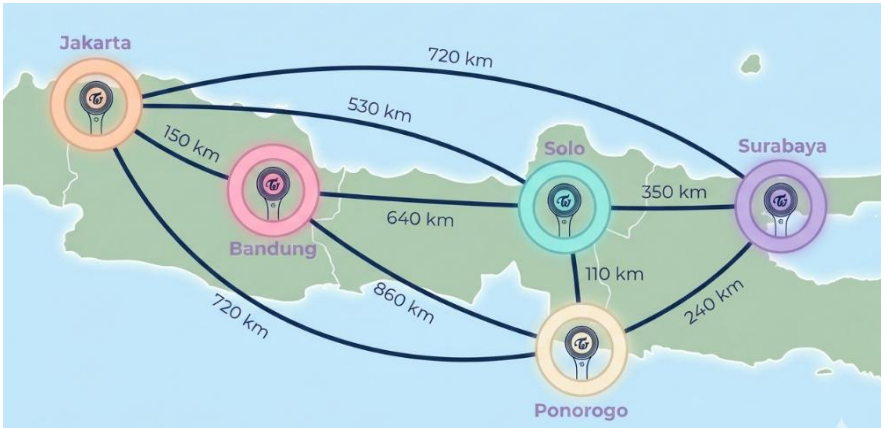
2.1. Studi Kasus: Rute Tur TWICE di Indonesia

Dalam konteks tugas ini, algoritma *Brute Force* diimplementasikan untuk mensimulasikan perencanaan rute konser World Tour TWICE di Indonesia, dengan fokus pada lima kota: Jakarta, Solo, Ponorogo, Bandung, dan Surabaya. Konteks studi kasus tur konser ini diambil dari jadwal dunia grup musik TWICE pada World Tour ke 5 mereka, READY TO BE (Sumber Inspirasi: https://en.wikipedia.org/wiki/Ready_to_Be_World_Tour).

Studi kasus dalam implementasi ini mengangkat skenario manajemen tur konser dunia TWICE di Indonesia. Kota-kota yang dipilih dalam graf adalah:

1. Jakarta (Pusat ibu kota dan bandara internasional utama)
2. Bandung (Kota metropolitan di Jawa Barat)
3. Solo (Kota budaya di Jawa Tengah)
4. Ponorogo (Kabupaten di Jawa Timur, dekat dengan Solo)
5. Surabaya (Kota metropolitan terbesar di Jawa Timur)

Pemilihan kelima kota ini mewakili sebaran geografis dari barat (Jakarta/Bandung) hingga timur (Ponorogo/Surabaya), dengan Solo sebagai titik tengah. Tantangan utamanya adalah menemukan urutan kunjungan yang menghindari perjalanan bolak-balik antara Jawa Barat dan Jawa Timur (Sumber: https://tarungrecords.wordpress.com/2018/01/23/panduan-rute-tur-band-ke-jawa-timur/).



Gambar 2. 1 Visualisasi Graph Rute World Tour TWICE untuk Studi Kasus TSP

2.2. Contoh Simulasi

Sebelum masuk ke implementasi kode program, dilakukan simulasi manual sederhana untuk memvalidasi logika algoritma *Brute Force* pada studi kasus tur konser TWICE. Simulasi ini melibatkan 5 kota, yakni Jakarta (A), Bandung (B), Solo (C), Ponorogo (D), dan Surabaya (E). Data jarak antar kota yang digunakan merupakan estimasi jarak darat yang divalidasi menggunakan peta digital (Link Sumber: https://psce.pw/8zezwg).

Dengan menetapkan Jakarta (A) sebagai titik awal dan akhir yang tetap, maka algoritma hanya perlu melakukan permutasi pada 4 kota sisanya (B, C, D, E). Data jarak diambil dari estimasi peta (https://maps.google.com) Jumlah total kemungkinan rute yang harus diperiksa adalah $(5-1)! = 4! = 24$ permutasi. Adapun data jarak antar kota berdasar graph adalah:

a) Jakarta – Bandung: 150 km
b) Jakarta – Solo: 530 km
c) Jakarta – Ponorogo: 720 km
d) Jakarta – Surabaya: 800 km
e) Bandung – Solo: 640 km
f) Bandung – Ponorogo: 860 km
g) Bandung – Surabaya: 750 km
h) Solo – Ponorogo: 110 km
i) Solo – Surabaya: 350 km
j) Ponorogo – Surabaya: 240 km

Berikut adalah tabel simulasi untuk ke-24 kemungkinan rute tersebut:

Tabel 2. 1 Simulasi Perhitungan Bobot Rute TSP

No	Rute	Perhitungan Bobot (km)	Total Jarak
1	A → B → C → D → E → A	150+640+110+240+800	1.940
2	A → B → C → E → D → A	150+640+350+240+720	2.100
3	A → B → D → C → E → A	150+860+110+350+800	2.270

No	Rute	Perhitungan Bobot (km)	Total Jarak
4	A → B → D → E → C → A	150+860+240+350+530	2.130
5	A → B → E → C → D → A	150+750+350+110+720	2.080
6	A → B → E → D → C → A	150+750+240+110+530	1.780
7	A → C → B → D → E → A	530+640+860+240+800	3.070
8	A → C → B → E → D → A	530+640+750+240+720	2.880
9	A → C → D → B → E → A	530+110+860+750+800	3.050
10	A → C → D → E → B → A	530+110+240+750+150	1.780
11	A → C → E → B → D → A	530+350+750+860+720	3.210
12	A → C → E → D → B → A	530+350+240+860+150	2.130
13	A → D → B → C → E → A	720+860+640+350+800	3.370
14	A → D → B → E → C → A	720+860+750+350+530	3.210
15	A → D → C → B → E → A	720+110+640+750+800	3.020
16	A → D → C → E → B → A	720+110+350+750+150	2.080
17	A → D → E → B → C → A	720+240+750+640+530	2.880
18	A → D → E → C → B → A	720+240+350+640+150	2.100
19	A → E → B → C → D → A	800+750+640+110+720	3.020
20	A → E → B → D → C → A	800+750+860+110+530	3.050
21	A → E → C → B → D → A	800+350+640+860+720	3.370
22	A → E → C → D → B → A	800+350+110+860+150	2.270
23	A → E → D → B → C → A	800+240+860+640+530	3.070
24	A → E → D → C → B → A	800+240+110+640+150	1.940

Berdasarkan tabel 2.1, dari 24 kemungkinan rute, ditemukan dua rute dengan total jarak minimum **1.780 km**, yaitu:

1. A → B → E → D → C → A (Jakarta → Bandung → Surabaya → Ponorogo → Solo → Jakarta)
2. A → C → D → E → B → A (Jakarta → Solo → Ponorogo → Surabaya → Bandung → Jakarta) *merupakan kebalikan dari rute no 1.*

Hal ini membuktikan bahwa strategi pengelompokan wilayah menghasilkan efisiensi jarak terbaik dibandingkan rute yang bolak-balik antar pulau Jawa.

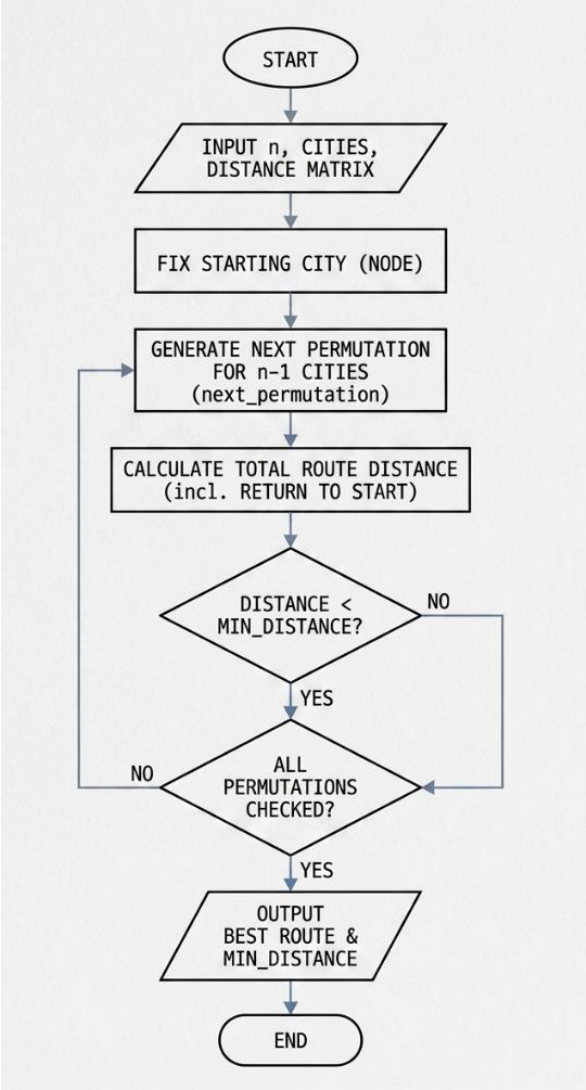
2.3. Data Jarak Antar Kota

Data jarak yang digunakan dalam simulasi ini adalah estimasi jarak tempuh melalui jalur darat (kilometer). Matriks jarak disajikan pada Tabel 3.1:

Tabel 3. 1 Matriks Jarak Antar Kota (dalam km)

Kota	Jakarta	Bandung	Solo	Ponorogo	Surabaya
Jakarta	0	150	530	720	800
Bandung	150	0	640	860	750
Solo	530	640	0	110	350
Ponorogo	720	860	110	0	240
Surabaya	800	750	350	240	0

Tabel 3.1 menunjukkan matriks jarak simetris antar 5 kota studi kasus. Nilai 0 pada diagonal utama menunjukkan jarak kota ke dirinya sendiri, sedangkan nilai lainnya merepresentasikan estimasi jarak darat (km). Data ini menjadi dasar perhitungan bobot (*weight*) dalam algoritma *Brute Force.* Untuk memperjelas mekanisme kerja pada kasus ini, berikut disajikan flowchart logika *Brute Force* yang diimplementasikan dalam program:



Gambar 2. 2 Flowchart Logika Algoritma Brute Force untuk TSP

Berdasarkan Gambar 2.2, terlihat bahwa algoritma bekerja secara iteratif membangkitkan setiap kemungkinan permutasi rute, menghitung bobotnya, dan hanya menyimpan rute jika ditemukan jarak yang lebih pendek dari iterasi sebelumnya. Proses ini berulang hingga seluruh $(n-1)!$ kemungkinan habis diperiksa, dan program C++ nanti akan otomatis menemukan nilai minimum ini.

2.4. Source Code Program (C++)

```
#include <iostream>
#include <vector>
#include <algorithm> // Untuk next_permutation
#include <climits>   // Untuk INT_MAX
using namespace std;
```

```
const int MAX_CITIES = 5;
string cityNames[MAX_CITIES] = {
    "Jakarta", "Bandung", "Solo", "Ponorogo", "Surabaya"
};

int distMatrix[MAX_CITIES][MAX_CITIES] = {
    // JKT  BDG   SOLO   PNG   SBY
    {0,     150,  530,  720,  800},   // JKT
    {150,   0,    640,  860,  750},   // BDG
    {530,   640,  0,    110,  350},   // SOLO
    {720,   860,  110,  0,    240},   // PNG
    {800,   750,  350,  240,  0}      // SBY
};

void showHeader() {
    cout << "==================================" << endl;
    cout << "IMPLEMENTASI ALGORITMA BRUTE FORCE PADA TSP"
<< endl;
    cout << "THIS IS FOR TWICE WORLD TOUR IN INDONESIA"
<< endl;
    cout << "----------------------------------" << endl;
    cout << "Nama  : Puji Astuti Suryaningtyas" << endl;
    cout << "NIM   : 24533867" << endl;
    cout << "Kelas : 4D Teknik Informatika" << endl;
    cout << "=========================" << endl << endl;
}

void showFooter() {
    cout << "\n================================" << endl;
    cout << "PROGRAM SELESAI DIIJALANKAN" << endl;
    cout << "----------------------------------" << endl;
    cout << "Program ini disusun oleh:" << endl;
    cout << "Nama  : Puji Astuti Suryaningtyas" << endl;
    cout << "NIM   : 24533867" << endl;
    cout << "Kelas : 4D Teknik Informatika" << endl;
    cout << "Universitas Muhammadiyah Ponorogo" << endl;
    cout << "==================================" << endl;
}

int main() {
    char choice;
    do {
        showHeader();
        cout << "Daftar Kota Tersedia:" << endl;
        for(int i=0; i<MAX_CITIES; i++) {
            cout << (i+1) << ". " << cityNames[i] << endl;
        }
```

```
        int n;
        cout << "\nMasukkan jumlah kota yang akan
dikunjungi (Maks 5): ";
        cin >> n;

        if(n < 2 || n > MAX_CITIES) {
            cout << "\n[ERROR] Jumlah kota harus antara 2
sampai 5!" << endl;
            cout << "Silakan coba lagi dengan jumlah yang
benar.\n" << endl;
            continue;
        }

        vector<int> cities;
        cout << endl;
        for(int i=0; i<n; i++) {
            int choiceCity;
            bool valid = false;
            while(!valid) {
                cout << "Pilih kota ke-" << (i+1) << " (1-
" << MAX_CITIES << "): ";
                cin >> choiceCity;

                if(choiceCity >= 1 && choiceCity <=
MAX_CITIES) {
                    bool exists = false;
                    for(int c : cities) {
                        if(c == choiceCity-1) exists =
true;
                    }

                    if(!exists) {
                        cities.push_back(choiceCity-1);
                        valid = true;
                    } else {
                        cout << "  -> Kota sudah dipilih!
Pilih kota lain.\n";
                    }
                } else {
                    cout << "  -> Input salah! Masukkan
angka 1-" << MAX_CITIES << "\n";
                }
            }
        }

        cout << "\nMenghitung semua kemungkinan rute ("
<< n << " kota)..." << endl;
```

```
        cout << "Mohon tunggu sebentar..." << endl;

        int startNode = cities[0];
        vector<int> others;
        for(size_t i=1; i<cities.size(); i++) {
            others.push_back(cities[i]);
        }

        sort(others.begin(), others.end());

        int minDist = INT_MAX;
        vector<int> bestRoute;

        do {
            vector<int> currentRoute;
            currentRoute.push_back(startNode);
            for(int        node        :        others)
currentRoute.push_back(node);

            int currentDist = 0;
            for(size_t i=0; i<currentRoute.size()-1;
i++) {
                currentDist +=
distMatrix[currentRoute[i]][currentRoute[i+1]];
            }
            currentDist +=
distMatrix[currentRoute.back()][startNode];

            if(currentDist < minDist) {
                minDist = currentDist;
                bestRoute = currentRoute;
            }

        }          while(next_permutation(others.begin(),
others.end()));

        cout << "\n----------------------------" << endl;
        cout << "HASIL OPTIMASI RUTE TERPENDEK:" << endl;
        cout << "-----------------------------" << endl;
        cout << "Rute Terbaik: ";
        for(size_t i=0; i<bestRoute.size(); i++) {
            cout << cityNames[bestRoute[i]];
            if(i < bestRoute.size()-1) cout << " -> ";
        }
        cout  <<  "  ->  "  <<  cityNames[startNode]  <<  "
(Kembali)" << endl;
```

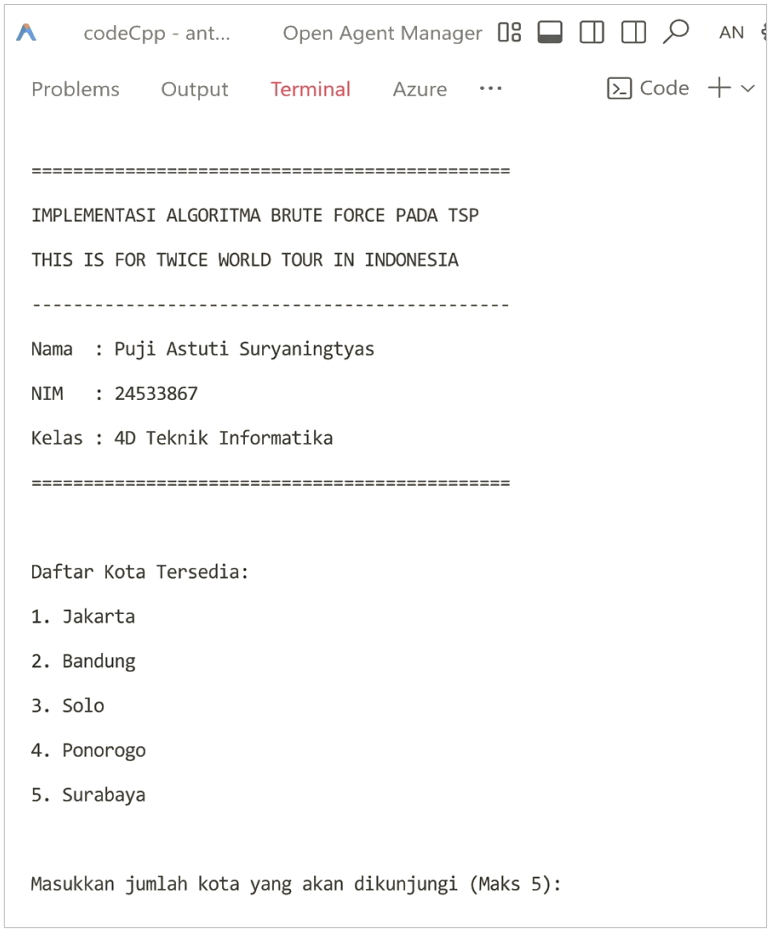
```
                cout << "Total Jarak Tempuh: " << minDist
<< " km" << endl;
        cout << "-----------------------------" << endl;
        cout << "\nApakah Anda ingin menghitung rute lain?
(Y/N): ";
        cin >> choice;
        cout << endl;

    } while (choice == 'Y' || choice == 'y');

    showFooter();
    return 0;
}
```

Implementasi permutasi rute pada kode di atas menggunakan fungsi standar library C++ yaitu std::next_permutation. (Link Sumber: https://en.cppreference.com/w/cpp/algorithm/next_permutation).

2.5. Screenshot Output Program



Gambar 3. 1 Tampilan Input dan Identitas Mahasiswa

Program pada Gambar 3.1 akan memproses input tersebut menggunakan algoritma *Brute Force* dan menampilkan rute terpendek beserta total jaraknya, berikut tangkapan layar hasil eksekusi program dengan skenario pengujian memilih kelima kota sekaligus.

Gambar 3. 2 Tampilan Output Program Optimasi Rute TSP dengan Input 5 Kota

Gambar 3.2 menunjukkan tampilan antarmuka konsol saat program dijalankan. Pengguna melakukan input dengan memilih 5 kota sekaligus. Setelah itu, program memproses 24 kemungkinan permutasi menggunakan algoritma Brute Force. Hasil eksekusi menampilkan rute terpendek yaitu **Jakarta → Bandung → Surabaya → Ponorogo → Solo → Jakarta** dengan total jarak tempuh **1.780 km**. Nilai ini konsisten dengan hasil simulasi manual pada Tabel 2.1.

SUMBER REFERENSI DAN INSPIRASI

Laporan dan program ini disusun dengan merujuk pada sumber-sumber berikut:

1. Konsep Logistik Event Musik: https://www.indonesia.travel/gb/en/events/event-detail/babymonster-1st-world-tour /
2. Teori Traveling Salesman Problem (TSP): https://terralogiq.com/traveling-salesman/
3. Analisis Algoritma Brute Force: https://jurnal.untan.ac.id/index.php/jbmstr/article/download/9608/9421
4. Visualisasi Algoritma TSP: https://youtu.be/gsYmZdZuFCw?si=oACj2tKPGRW7r9sJ
5. Jadwal Tur TWICE "Ready to Be": https://en.wikipedia.org/wiki/Ready_to_Be_World_Tour
6. Referensi Rute Wilayah Jawa Timur: https://tarungrecords.wordpress.com/2018/01/23/panduan-rute-tur-band-ke-jawa-timur/
7. Estimasi Jarak Antar Kota (Google Maps): https://psce.pw/8zezwg dan https://maps.google.com
8. Dokumentasi Fungsi C++ next_permutation: https://en.cppreference.com/w/cpp/algorithm/next_permutation